

Justificativa da Arquitetura de Software

Sistema de Gerenciamento de Jogadores de Valorant

Introdução

Antes de iniciar o desenvolvimento, é necessário definir o estilo arquitetural que vai organizar o sistema. Duas opções foram avaliadas: a arquitetura em camadas (layered architecture) e a arquitetura em pipeline. Este documento explica as características de cada uma e justifica a escolha da arquitetura em camadas para o desenvolvimento do sistema de gerenciamento de jogadores de Valorant.

Arquitetura em pipeline

A arquitetura em pipeline organiza o processamento como uma sequência fixa de estágios, em que a saída de um estágio é a entrada do próximo. Cada estágio executa uma transformação específica sobre os dados, e o fluxo é predominantemente unidirecional. Esse modelo é muito utilizado em processos de ETL (extração, transformação e carga de dados), processamento de imagens ou vídeo, compiladores e rotinas de processamento em lote (batch), em que existe uma única sequência de passos bem definida do início ao fim.

O ponto central desse estilo é que ele foi pensado para transformar um conjunto de dados de entrada em uma saída final, por meio de etapas encadeadas — e não para atender, de forma independente, dezenas de operações distintas solicitadas por diferentes usuários a qualquer momento.

Arquitetura em camadas

A arquitetura em camadas organiza o sistema em camadas horizontais, cada uma com uma responsabilidade bem definida, e a comunicação ocorre normalmente apenas entre camadas adjacentes. Para este projeto, propõem-se três camadas, alinhadas às tecnologias já definidas nos requisitos não funcionais:

- Camada de apresentação (frontend): HTML, CSS e JavaScript, responsável pelas telas de login, cadastro, pesquisa, dashboard e demais interfaces com o usuário.
- Camada de aplicação / regras de negócio (backend): PHP, responsável pela autenticação, validações, controle de acesso por perfil, cálculo de estatísticas e geração de rankings.
- Camada de dados: banco de dados MySQL, administrado via phpMyAdmin, responsável pelo armazenamento de usuários, jogadores, estatísticas e demais informações persistidas.

Cada requisição do usuário passa pelas mesmas três camadas, mas aciona operações diferentes dentro delas — por exemplo, uma pesquisa de jogador e um cadastro de jogador usam a mesma camada de dados, mas executam consultas e regras de negócio distintas.

Comparação entre os dois estilos

Critério	Arquitetura em camadas	Arquitetura em pipeline
Natureza do fluxo	Requisições independentes, acessadas em qualquer ordem pelo usuário	Sequência fixa de estágios, um alimentando o próximo
Reuso de componentes	Alto: várias funcionalidades reutilizam as mesmas camadas de dados e de negócio	Baixo: cada pipeline costuma ser desenhado para uma transformação específica
Controle de acesso por perfil	Centralizado na camada de aplicação, aplicado a qualquer operação	Precisaria ser replicado em cada estágio do fluxo
Manutenibilidade	Camadas isoladas: trocar o banco ou a interface não afeta as demais	Alterar um estágio pode impactar os estágios seguintes
Adequação a aplicações web CRUD	Alta: é o padrão natural para sistemas com login, cadastros, consultas e relatórios	Baixa: mais indicada para ETL, processamento em lote ou transformação de arquivos

Justificativa da escolha

A arquitetura em camadas foi escolhida pelos seguintes motivos, todos relacionados às características do sistema descritas nos requisitos funcionais e não funcionais:

- Funcionalidades independentes: o sistema reúne 15 requisitos funcionais (login, gestão de usuários, cadastro e importação de jogadores, pesquisas, filtros, estatísticas, ranking, dashboard, entre outros) que são acessados pelo usuário em qualquer ordem, e não como etapas fixas de um único fluxo — o que corresponde ao modelo de camadas, e não ao de pipeline.
- Reuso de componentes: consultas a jogadores são reutilizadas por várias funcionalidades (RF05, RF06, RF07, RF08, RF09, RF13, RF14 e RF15). Na arquitetura em camadas, essa lógica fica centralizada na camada de dados e de negócio e é reaproveitada; em um pipeline, cada fluxo tende a ser desenhado de forma isolada, dificultando esse reuso.
- Controle de acesso por perfil (RNF11): como administrador e usuário comum têm permissões diferentes, é mais simples concentrar essa verificação na camada de aplicação do que replicá-la em múltiplos estágios de um pipeline.

- Manutenibilidade (RNF08): a separação em camadas permite alterar uma parte do sistema (por exemplo, trocar a forma como o front-end exibe os dados) sem impactar as demais camadas, o que facilita correções e evoluções futuras.
- Aderência às tecnologias definidas (RNF10 e RNF09): a divisão HTML/CSS/JavaScript, PHP e MySQL/phpMyAdmin já mapeia naturalmente para as camadas de apresentação, aplicação e dados.

A arquitetura em pipeline até poderia descrever, de forma pontual, o fluxo interno da importação de jogadores por planilha Excel (RF04) — leitura do arquivo, validação e gravação no banco — mas isso representa apenas uma funcionalidade específica, não a arquitetura do sistema como um todo. Adotar pipeline como arquitetura geral exigiria forçar todas as demais funcionalidades (login, pesquisas, dashboards, rankings) em um formato de estágios sequenciais, o que não reflete a forma como esses recursos são utilizados.